```
/**
 * 应用主入口文件
 * 负责创建 Vue 应用实例，注册全局插件和配置
 */
import { createApp } from 'vue'
import App from './App.vue'
import router from './router'
import { createPinia } from 'pinia'
import ElementPlus from 'element-plus'
import 'element-plus/dist/index.css'
import zhCn from 'element-plus/es/locale/lang/zh-cn'
import * as ElementPlusIconsVue from '@element-plus/icons-vue'
import axios from 'axios'
import './styles/global.css'
// 创建 axios 实例，配置基础 URL 和超时时间
const axiosInstance = axios.create({
  baseURL: process.env.VUE_APP_API_BASE_URL || 'http://localhost:5000/api',
  timeout: 30000,
  headers: {
    'Content-Type': 'application/json'
  }
})
// 请求拦截器：自动添加 JWT Token 到请求头
axiosInstance.interceptors.request.use(
  config => {
    const token = localStorage.getItem('token')
    if (token) {
      config.headers['Authorization'] = `Bearer ${token}`
    }
    return config
  },
  error => {
    return Promise.reject(error)
  }
)
// 响应拦截器：统一处理错误响应
axiosInstance.interceptors.response.use(
  response => {
    return response.data
  },
  error => {
    if (error.response) {
      switch (error.response.status) {
        case 401:
          // Token 过期或未授权，跳转到登录页
          localStorage.removeItem('token')
          localStorage.removeItem('userInfo')
          window.location.href = '/login'
          break
        case 403:
```

```
          ElementPlus.ElMessage.error('没有权限访问该资源')
          break
        case 404:
          ElementPlus.ElMessage.error('请求的资源不存在')
          break
        case 500:
          ElementPlus.ElMessage.error('服务器内部错误，请稍后重试')
          break
        default:
          ElementPlus.ElMessage.error(error.response.data.message || '请求失败')
      }
    } else if (error.request) {
      ElementPlus.ElMessage.error('网络连接失败，请检查网络设置')
    } else {
      ElementPlus.ElMessage.error('请求配置错误')
    }
    return Promise.reject(error)
  }
)
// 创建 Vue 应用实例
const app = createApp(App)
// 创建 Pinia 状态管理实例
const pinia = createPinia()
// 注册 Element Plus 图标组件
for (const [key, component] of Object.entries(ElementPlusIconsVue)) {
  app.component(key, component)
}
// 注册全局插件
app.use(router)
app.use(pinia)
app.use(ElementPlus, {
  locale: zhCn
})
// 将 axios 实例挂载到全局属性，方便组件中使用
app.config.globalProperties.$axios = axiosInstance
// 全局错误处理
app.config.errorHandler = (err, instance, info) => {
  console.error('全局错误:', err)
  console.error('错误信息:', info)
  ElementPlus.ElMessage.error('程序发生错误，请刷新页面重试')
}
// 挂载应用
app.mount('#app')
<template>
  <div id="app">
    <!-- 顶部导航栏 -->
    <el-header class="app-header" v-if="!isLoginPage">
      <div class="header-left">
        <h1 class="app-title">传统康普茶数字化生产管理平台 V1.0</h1>
      </div>
```

```
    <div class="header-right">
      <el-dropdown @command="handleUserCommand">
        <span class="user-info">
          <el-avatar :size="36" :src="userInfo.avatar">
            {{ userInfo.realName ? userInfo.realName.substring(0, 1) : '用' }}
          </el-avatar>
          <span class="user-name">{{ userInfo.realName || '用户' }}</span>
          <el-icon><ArrowDown /></el-icon>
        </span>
        <template #dropdown>
          <el-dropdown-menu>
            <el-dropdown-item command="profile">
              <el-icon><User /></el-icon>个人中心
            </el-dropdown-item>
            <el-dropdown-item command="settings">
              <el-icon><Setting /></el-icon>系统设置
            </el-dropdown-item>
            <el-dropdown-item command="logout" divided>
              <el-icon><SwitchButton /></el-icon>退出登录
            </el-dropdown-item>
          </el-dropdown-menu>
        </template>
      </el-dropdown>
    </div>
  </el-header>
  <!-- 主体内容区域 -->
  <div class="app-container" v-if="!isLoginPage">
    <!-- 侧边栏导航菜单 -->
    <el-aside :width="isCollapse ? '64px' : '240px'" class="app-aside">
      <el-menu
        :default-active="activeMenu"
        :collapse="isCollapse"
        :unique-opened="true"
        router
        class="aside-menu"
      >
        <el-menu-item index="/materials">
          <el-icon><Box /></el-icon>
          <template #title>原料管理</template>
        </el-menu-item>
        <el-menu-item index="/fermentation">
          <el-icon><Operation /></el-icon>
          <template #title>发酵监控</template>
        </el-menu-item>
        <el-menu-item index="/batches">
          <el-icon><Search /></el-icon>
          <template #title>批次追踪</template>
        </el-menu-item>
        <el-menu-item index="/inventory">
          <el-icon><DataAnalysis /></el-icon>
```

```
          <template #title>库存管理</template>
        </el-menu-item>
        <el-menu-item index="/monitoring">
          <el-icon><Monitor /></el-icon>
          <template #title>实时监控</template>
        </el-menu-item>
        <el-menu-item index="/optimization">
          <el-icon><TrendCharts /></el-icon>
          <template #title>工艺优化</template>
        </el-menu-item>
        <el-menu-item index="/tasks">
          <el-icon><List /></el-icon>
          <template #title>任务中心</template>
        </el-menu-item>
      </el-menu>
      <div class="collapse-btn" @click="toggleCollapse">
        <el-icon>
          <Fold v-if="!isCollapse" />
          <Expand v-else />
        </el-icon>
      </div>
    </el-aside>
    <!-- 主内容区域 -->
    <el-main class="app-main">
      <router-view v-slot="{ Component }">
        <transition name="fade" mode="out-in">
          <component :is="Component" />
        </transition>
      </router-view>
    </el-main>
  </div>
  <!-- 登录页面单独渲染 -->
  <router-view v-else />
 </div>
</template>
<script setup>
import { ref, computed, onMounted } from 'vue'
import { useRouter, useRoute } from 'vue-router'
import { useUserStore } from '@/stores/user'
import { ElMessageBox, ElMessage } from 'element-plus'
// 路由实例
const router = useRouter()
const route = useRoute()
// 用户状态管理
const userStore = useUserStore()
// 响应式数据
const isCollapse = ref(false)
// 计算属性：判断是否为登录页面
const isLoginPage = computed(() => {
  return route.path === '/login'
```

```
})
// 计算属性：当前激活的菜单项
const activeMenu = computed(() => {
  return route.path
})
// 计算属性：用户信息
const userInfo = computed(() => {
  return userStore.userInfo || {}
})
// 方法：切换侧边栏折叠状态
const toggleCollapse = () => {
  isCollapse.value = !isCollapse.value
}
// 方法：处理用户下拉菜单命令
const handleUserCommand = async (command) => {
  switch (command) {
    case 'profile':
      router.push('/profile')
      break
    case 'settings':
      router.push('/settings')
      break
    case 'logout':
      try {
        await ElMessageBox.confirm('确定要退出登录吗？', '提示', {
          confirmButtonText: '确定',
          cancelButtonText: '取消',
          type: 'warning'
        })
        await userStore.logout()
        ElMessage.success('退出登录成功')
        router.push('/login')
      } catch (error) {
        // 用户取消操作
      }
      break
  }
}
// 生命周期：组件挂载后检查登录状态
onMounted(async () => {
  if (!isLoginPage.value) {
    try {
      await userStore.getUserInfo()
    } catch (error) {
      console.error('获取用户信息失败:', error)
      router.push('/login')
    }
  }
})
</script>
```

```
<style scoped>
#app {
  width: 100%;
  height: 100vh;
  overflow: hidden;
}
.app-header {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0 20px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}
.header-left .app-title {
  font-size: 20px;
  font-weight: 500;
  margin: 0;
}
.header-right .user-info {
  display: flex;
  align-items: center;
  gap: 10px;
  cursor: pointer;
  padding: 5px 10px;
  border-radius: 4px;
  transition: background-color 0.3s;
}
.header-right .user-info:hover {
  background-color: rgba(255, 255, 255, 0.1);
}
.user-name {
  font-size: 14px;
}
.app-container {
  display: flex;
  height: calc(100vh - 60px);
}
.app-aside {
  background-color: #ffffff;
  border-right: 1px solid #e8e8e8;
  overflow-x: hidden;
  transition: width 0.3s;
}
.aside-menu {
  border-right: none;
  height: calc(100% - 40px);
}
.collapse-btn {
```

```
  height: 40px;
  display: flex;
  align-items: center;
  justify-content: center;
  cursor: pointer;
  border-top: 1px solid #e8e8e8;
  color: #909399;
  transition: all 0.3s;
}
.collapse-btn:hover {
  background-color: #f5f7fa;
  color: #409EFF;
}
.app-main {
  background-color: #f0f2f5;
  padding: 20px;
  overflow-y: auto;
}
/* 路由切换动画 */
.fade-enter-active,
.fade-leave-active {
  transition: opacity 0.3s;
}
.fade-enter-from,
.fade-leave-to {
  opacity: 0;
}
</style>
/**
 * 路由配置文件
 * 定义应用的所有路由及其对应的组件
 */
import { createRouter, createWebHistory } from 'vue-router'
import { useUserStore } from '@/stores/user'
// 路由配置数组
const routes = [
  {
    path: '/login',
    name: 'Login',
    component: () => import('@/views/Login.vue'),
    meta: {
      title: '登录',
      requiresAuth: false
    }
  },
  {
    path: '/',
    redirect: '/materials'
  },
  {
```

```
  path: '/materials',
  name: 'Materials',
  component: () => import('@/views/Materials/Index.vue'),
  meta: {
    title: '原料管理',
    requiresAuth: true,
    icon: 'Box'
  }
},
{
  path: '/fermentation',
  name: 'Fermentation',
  component: () => import('@/views/Fermentation/Index.vue'),
  meta: {
    title: '发酵监控',
    requiresAuth: true,
    icon: 'Operation'
  }
},
{
  path: '/fermentation/create',
  name: 'FermentationCreate',
  component: () => import('@/views/Fermentation/Create.vue'),
  meta: {
    title: '创建发酵批次',
    requiresAuth: true
  }
},
{
  path: '/fermentation/:id',
  name: 'FermentationDetail',
  component: () => import('@/views/Fermentation/Detail.vue'),
  meta: {
    title: '发酵批次详情',
    requiresAuth: true
  }
},
{
  path: '/batches',
  name: 'Batches',
  component: () => import('@/views/Batches/Index.vue'),
  meta: {
    title: '批次追踪',
    requiresAuth: true,
    icon: 'Search'
  }
},
{
  path: '/batches/:id',
  name: 'BatchDetail',
```

```
    component: () => import('@/views/Batches/Detail.vue'),
    meta: {
      title: '批次详情',
      requiresAuth: true
    }
  },
  {
    path: '/inventory',
    name: 'Inventory',
    component: () => import('@/views/Inventory/Index.vue'),
    meta: {
      title: '库存管理',
      requiresAuth: true,
      icon: 'DataAnalysis'
    }
  },
  {
    path: '/monitoring',
    name: 'Monitoring',
    component: () => import('@/views/Monitoring/Index.vue'),
    meta: {
      title: '实时监控',
      requiresAuth: true,
      icon: 'Monitor'
    }
  },
  {
    path: '/optimization',
    name: 'Optimization',
    component: () => import('@/views/Optimization/Index.vue'),
    meta: {
      title: '工艺优化',
      requiresAuth: true,
      icon: 'TrendCharts'
    }
  },
  {
    path: '/tasks',
    name: 'Tasks',
    component: () => import('@/views/Tasks/Index.vue'),
    meta: {
      title: '任务中心',
      requiresAuth: true,
      icon: 'List'
    }
  },
  {
    path: '/profile',
    name: 'Profile',
    component: () => import('@/views/Profile.vue'),
```

```
    meta: {
      title: '个人中心',
      requiresAuth: true
    }
  },
  {
    path: '/settings',
    name: 'Settings',
    component: () => import('@/views/Settings.vue'),
    meta: {
      title: '系统设置',
      requiresAuth: true
    }
  },
  {
    path: '/:pathMatch(.*)*',
    name: 'NotFound',
    component: () => import('@/views/NotFound.vue'),
    meta: {
      title: '页面不存在'
    }
  }
]
// 创建路由实例
const router = createRouter({
  history: createWebHistory(process.env.VUE_APP_BASE_URL || '/'),
  routes
})
// 全局前置守卫：检查路由权限
router.beforeEach((to, from, next) => {
  // 设置页面标题
  document.title = to.meta.title ? `${to.meta.title} - 传统康普茶数字化生产管理平台` : '传统康普茶数字化生产管理平台'
  // 检查是否需要登录
  const userStore = useUserStore()
  const isAuthenticated = !!localStorage.getItem('token')
  if (to.meta.requiresAuth && !isAuthenticated) {
    // 需要登录但未登录，跳转到登录页
    next({
      path: '/login',
      query: { redirect: to.fullPath }
    })
  } else if (to.path === '/login' && isAuthenticated) {
    // 已登录用户访问登录页，跳转到首页
    next({ path: '/' })
  } else {
    next()
  }
})
// 全局后置钩子：完成导航后执行
```

```
router.afterEach((to, from) => {
  // 可以在这里添加页面访问统计等逻辑
  console.log(`路由跳转: ${from.path} -> ${to.path}`)
})
export default router
/**
 * API 基础配置文件
 * 导出所有 API 模块
 */
import axios from 'axios'
// 创建 axios 实例
const service = axios.create({
  baseURL: process.env.VUE_APP_API_BASE_URL || 'http://localhost:5000/api',
  timeout: 30000
})
// 请求拦截器
service.interceptors.request.use(
  config => {
    const token = localStorage.getItem('token')
    if (token) {
      config.headers['Authorization'] = `Bearer ${token}`
    }
    return config
  },
  error => {
    console.error('请求错误:', error)
    return Promise.reject(error)
  }
)
// 响应拦截器
service.interceptors.response.use(
  response => {
    const res = response.data
    if (res.code !== 200) {
      console.error('响应错误:', res.message)
      return Promise.reject(new Error(res.message || 'Error'))
    } else {
      return res
    }
  },
  error => {
    console.error('响应错误:', error)
    return Promise.reject(error)
  }
)
export default service
/**
 * 用户相关 API 接口
 */
import request from './index'
```

```
/**
 * 用户登录
 * @param {Object} data - 登录信息 {username, password}
 * @returns {Promise} 返回包含 token 和用户信息的响应
 */
export function login(data) {
  return request({
    url: '/users/login',
    method: 'post',
    data
  })
}
/**
 * 用户登出
 * @returns {Promise} 返回登出结果
 */
export function logout() {
  return request({
    url: '/users/logout',
    method: 'post'
  })
}
/**
 * 获取当前用户信息
 * @returns {Promise} 返回用户详细信息
 */
export function getUserInfo() {
  return request({
    url: '/users/me',
    method: 'get'
  })
}
/**
 * 修改密码
 * @param {Object} data - 密码信息 {oldPassword, newPassword}
 * @returns {Promise} 返回修改结果
 */
export function changePassword(data) {
  return request({
    url: '/users/change-password',
    method: 'put',
    data
  })
}
/**
 * 更新用户信息
 * @param {Object} data - 用户信息
 * @returns {Promise} 返回更新结果
 */
export function updateUserInfo(data) {
```

```
  return request({
    url: '/users/me',
    method: 'put',
    data
  })
}
/**
 * 原料管理相关 API 接口
 */
import request from './index'
/**
 * 获取原料列表
 * @param {Object} params - 查询参数 {page, size, type, category, keyword}
 * @returns {Promise} 返回原料列表和分页信息
 */
export function getMaterials(params) {
  return request({
    url: '/materials',
    method: 'get',
    params
  })
}
/**
 * 获取原料详情
 * @param {String} id - 原料 ID
 * @returns {Promise} 返回原料详细信息
 */
export function getMaterialDetail(id) {
  return request({
    url: `/materials/${id}`,
    method: 'get'
  })
}
/**
 * 新增原料
 * @param {Object} data - 原料信息
 * @returns {Promise} 返回新增的原料信息
 */
export function createMaterial(data) {
  return request({
    url: '/materials',
    method: 'post',
    data
  })
}
/**
 * 更新原料信息
 * @param {String} id - 原料 ID
 * @param {Object} data - 更新的原料信息
 * @returns {Promise} 返回更新结果
```

```
 */
export function updateMaterial(id, data) {
  return request({
    url: `/materials/${id}`,
    method: 'put',
    data
  })
}
/**
 * 删除原料
 * @param {String} id - 原料 ID
 * @returns {Promise} 返回删除结果
 */
export function deleteMaterial(id) {
  return request({
    url: `/materials/${id}`,
    method: 'delete'
  })
}
/**
 * 批量导入原料
 * @param {FormData} data - 包含 Excel 文件的 FormData 对象
 * @returns {Promise} 返回导入结果
 */
export function importMaterials(data) {
  return request({
    url: '/materials/import',
    method: 'post',
    data,
    headers: {
      'Content-Type': 'multipart/form-data'
    }
  })
}
/**
 * 导出原料数据
 * @param {Object} params - 查询参数
 * @returns {Promise} 返回 Excel 文件
 */
export function exportMaterials(params) {
  return request({
    url: '/materials/export',
    method: 'get',
    params,
    responseType: 'blob'
  })
}
/**
 * 获取原料分类列表
 * @returns {Promise} 返回分类树结构
```

```
 */
export function getMaterialCategories() {
  return request({
    url: '/materials/categories',
    method: 'get'
  })
}
/**
 * 上传原料附件
 * @param {String} id - 原料 ID
 * @param {FormData} data - 包含附件文件的 FormData 对象
 * @returns {Promise} 返回上传结果
 */
export function uploadMaterialAttachment(id, data) {
  return request({
    url: `/materials/${id}/attachments`,
    method: 'post',
    data,
    headers: {
      'Content-Type': 'multipart/form-data'
    }
  })
}
/**
 * 发酵监控相关 API 接口
 */
import request from './index'
/**
 * 获取发酵批次列表
 * @param {Object} params - 查询参数 {page, size, status, startDate, endDate}
 * @returns {Promise} 返回批次列表和分页信息
 */
export function getFermentationBatches(params) {
  return request({
    url: '/fermentation/batches',
    method: 'get',
    params
  })
}
/**
 * 获取发酵批次详情
 * @param {String} id - 批次 ID
 * @returns {Promise} 返回批次详细信息
 */
export function getFermentationBatchDetail(id) {
  return request({
    url: `/fermentation/batches/${id}`,
    method: 'get'
  })
}
```

```
/**
 * 创建发酵批次
 * @param {Object} data - 批次信息
 * @returns {Promise} 返回创建的批次信息
 */
export function createFermentationBatch(data) {
  return request({
    url: '/fermentation/batches',
    method: 'post',
    data
  })
}
/**
 * 更新发酵批次信息
 * @param {String} id - 批次 ID
 * @param {Object} data - 更新的批次信息
 * @returns {Promise} 返回更新结果
 */
export function updateFermentationBatch(id, data) {
  return request({
    url: `/fermentation/batches/${id}`,
    method: 'put',
    data
  })
}
/**
 * 录入每日监控数据
 * @param {String} id - 批次 ID
 * @param {Object} data - 监控数据
 * @returns {Promise} 返回录入结果
 */
export function recordFermentationData(id, data) {
  return request({
    url: `/fermentation/batches/${id}/records`,
    method: 'post',
    data
  })
}
/**
 * 获取批次监控记录
 * @param {String} id - 批次 ID
 * @param {Object} params - 查询参数 {page, size, startDate, endDate}
 * @returns {Promise} 返回监控记录列表
 */
export function getFermentationRecords(id, params) {
  return request({
    url: `/fermentation/batches/${id}/records`,
    method: 'get',
    params
  })
```

```
}
/**
 * 获取批次数据图表
 * @param {String} id - 批次 ID
 * @returns {Promise} 返回图表数据
 */
export function getFermentationChart(id) {
  return request({
    url: `/fermentation/batches/${id}/chart`,
    method: 'get'
  })
}
/**
 * 创建第二发酵批次
 * @param {Object} data - 第二发酵信息
 * @returns {Promise} 返回创建的第二发酵批次信息
 */
export function createSecondFermentation(data) {
  return request({
    url: '/fermentation/second-stage',
    method: 'post',
    data
  })
}
/**
 * 获取实时监控数据
 * @param {String} batchId - 批次 ID
 * @returns {Promise} 返回实时数据
 */
export function getRealtimeData(batchId) {
  return request({
    url: `/fermentation/realtime/${batchId}`,
    method: 'get'
  })
}
/**
 * 获取设备报警列表
 * @param {Object} params - 查询参数 {page, size, level, status}
 * @returns {Promise} 返回报警列表
 */
export function getAlerts(params) {
  return request({
    url: '/fermentation/alerts',
    method: 'get',
    params
  })
}
/**
 * 处理报警
 * @param {String} id - 报警 ID
```

```
 * @param {Object} data - 处理信息 {handleResult, handleNote}
 * @returns {Promise} 返回处理结果
 */
export function handleAlert(id, data) {
  return request({
    url: `/fermentation/alerts/${id}/handle`,
    method: 'post',
    data
  })
}
/**
 * 绑定 IoT 设备到批次
 * @param {Object} data - 绑定信息 {deviceId, batchId}
 * @returns {Promise} 返回绑定结果
 */
export function bindDevice(data) {
  return request({
    url: '/fermentation/devices/bind',
    method: 'post',
    data
  })
}
/**
 * 获取设备数据
 * @param {String} id - 设备 ID
 * @returns {Promise} 返回设备数据
 */
export function getDeviceData(id) {
  return request({
    url: `/fermentation/devices/${id}/data`,
    method: 'get'
  })
}
/**
 * 用户状态管理
 * 使用 Pinia 进行全局状态管理
 */
import { defineStore } from 'pinia'
import { login as loginApi, logout as logoutApi, getUserInfo as getUserInfoApi } from
'@/api/user'
import { ElMessage } from 'element-plus'
export const useUserStore = defineStore('user', {
  state: () => ({
    token: localStorage.getItem('token') || '',
    userInfo: JSON.parse(localStorage.getItem('userInfo') || '{}')
  }),
  getters: {
    /**
     * 判断是否已登录
     */
```

```
  isLoggedIn: (state) => !!state.token,
  /**
   * 获取用户真实姓名
   */
  realName: (state) => state.userInfo.realName || '',
  /**
   * 获取用户角色
   */
  roles: (state) => state.userInfo.roles || []
},
actions: {
  /**
   * 用户登录
   * @param {Object} loginForm - 登录表单数据 {username, password}
   */
  async login(loginForm) {
    try {
      const response = await loginApi(loginForm)
      this.token = response.data.token
      this.userInfo = response.data.userInfo
      // 保存到本地存储
      localStorage.setItem('token', this.token)
      localStorage.setItem('userInfo', JSON.stringify(this.userInfo))
      ElMessage.success('登录成功')
      return response
    } catch (error) {
      ElMessage.error(error.message || '登录失败')
      throw error
    }
  },
  /**
   * 用户登出
   */
  async logout() {
    try {
      await logoutApi()
    } catch (error) {
      console.error('登出失败:', error)
    } finally {
      // 清除本地存储
      this.token = ''
      this.userInfo = {}
      localStorage.removeItem('token')
      localStorage.removeItem('userInfo')
    }
  },
  /**
   * 获取用户信息
   */
  async getUserInfo() {
```

```
      try {
        const response = await getUserInfoApi()
        this.userInfo = response.data
        localStorage.setItem('userInfo', JSON.stringify(this.userInfo))
        return response
      } catch (error) {
        ElMessage.error('获取用户信息失败')
        throw error
      }
    },
    /**
     * 更新用户信息
     * @param {Object} data - 用户信息
     */
    updateUserInfo(data) {
      this.userInfo = { ...this.userInfo, ...data }
      localStorage.setItem('userInfo', JSON.stringify(this.userInfo))
    }
  }
})
<template>
  <div class="materials-page">
    <!-- 页面头部 -->
    <div class="page-header">
      <h2 class="page-title">原料信息管理</h2>
      <div class="header-actions">
        <el-button @click="handleImport">批量导入</el-button>
        <el-button @click="handleExport">导出</el-button>
        <el-button type="primary" @click="handleCreate">新增原料</el-button>
      </div>
    </div>
    <!-- 分类树和原料列表 -->
    <el-row :gutter="20">
      <!-- 左侧分类树 -->
      <el-col :span="6">
        <el-card class="category-card">
          <template #header>
            <div class="card-header">
              <span>原料分类</span>
              <el-button link type="primary" @click="handleRefreshCategories">
                <el-icon><Refresh /></el-icon>
              </el-button>
            </div>
          </template>
          <el-tree
            ref="categoryTreeRef"
            :data="categories"
            :props="treeProps"
            :highlight-current="true"
            node-key="id"
```

```
      @node-click="handleCategoryClick"
    >
      <template #default="{ node, data }">
        <span class="tree-node">
          <el-icon>
            <component :is="getCategoryIcon(data.type)" />
          </el-icon>
          <span>{{ node.label }}</span>
          <span class="count">({{ data.count }})</span>
        </span>
      </template>
    </el-tree>
  </el-card>
</el-col>
<!-- 右侧原料列表 -->
<el-col :span="18">
  <el-card class="list-card">
    <!-- 搜索栏 -->
    <div class="search-bar">
      <el-input
        v-model="searchForm.keyword"
        placeholder="请输入原料名称或编号"
        clearable
        style="width: 300px; margin-right: 10px;"
        @keyup.enter="handleSearch"
      >
        <template #prefix>
          <el-icon><Search /></el-icon>
        </template>
      </el-input>
      <el-select
        v-model="searchForm.type"
        placeholder="原料类型"
        clearable
        style="width: 150px; margin-right: 10px;"
      >
        <el-option label="茶叶" value="tea" />
        <el-option label="糖类" value="sugar" />
        <el-option label="菌种 SCOBY" value="scooby" />
        <el-option label="调味原料" value="flavor" />
        <el-option label="包装材料" value="packaging" />
      </el-select>
      <el-button type="primary" @click="handleSearch">
        <el-icon><Search /></el-icon> 查询
      </el-button>
      <el-button @click="handleReset">
        <el-icon><RefreshLeft /></el-icon> 重置
      </el-button>
    </div>
    <!-- 数据表格 -->
```

```
<el-table
  v-loading="loading"
  :data="materialList"
  stripe
  style="width: 100%; margin-top: 20px;"
>
  <el-table-column type="selection" width="55" />
  <el-table-column prop="materialId" label="原料编号" width="150" />
  <el-table-column prop="name" label="原料名称" width="150" />
  <el-table-column prop="type" label="类型" width="120">
    <template #default="{ row }">
      <el-tag :type="getTypeTagColor(row.type)">
        {{ getTypeLabel(row.type) }}
      </el-tag>
    </template>
  </el-table-column>
  <el-table-column prop="origin" label="产地" width="150" />
  <el-table-column prop="supplierName" label="供应商" width="150" />
  <el-table-column prop="stock" label="库存数量" width="120" align="right">
    <template #default="{ row }">
      <span :class="getStockClass(row.stock, row.safetyStock)">
        {{ row.stock }} {{ row.unit }}
      </span>
    </template>
  </el-table-column>
  <el-table-column prop="unitPrice" label="单价" width="100" align="right">
    <template #default="{ row }">
      ¥{{ row.unitPrice.toFixed(2) }}
    </template>
  </el-table-column>
  <el-table-column prop="status" label="状态" width="100">
    <template #default="{ row }">
      <el-tag :type="getStatusTagColor(row.status)">
        {{ getStatusLabel(row.status) }}
      </el-tag>
    </template>
  </el-table-column>
  <el-table-column label="操作" width="200" fixed="right">
    <template #default="{ row }">
      <el-button link type="primary" size="small" @click="handleView(row)">
        查看
      </el-button>
      <el-button link type="primary" size="small" @click="handleEdit(row)">
        编辑
      </el-button>
      <el-button link type="danger" size="small" @click="handleDelete(row)">
        删除
      </el-button>
    </template>
  </el-table-column>
```

```
        </el-table>
        <!-- 分页器 -->
        <el-pagination
          v-model:current-page="pagination.page"
          v-model:page-size="pagination.size"
          :page-sizes="[10, 20, 50, 100]"
          :total="pagination.total"
          layout="total, sizes, prev, pager, next, jumper"
          style="margin-top: 20px; justify-content: flex-end;"
          @size-change="handleSizeChange"
          @current-change="handleCurrentChange"
        />
      </el-card>
    </el-col>
  </el-row>
  <!-- 原料表单对话框 -->
  <el-dialog
    v-model="dialogVisible"
    :title="dialogTitle"
    width="800px"
    @close="handleDialogClose"
  >
    <el-form
      ref="materialFormRef"
      :model="materialForm"
      :rules="formRules"
      label-width="120px"
    >
      <el-row :gutter="20">
        <el-col :span="12">
          <el-form-item label="原料名称" prop="name">
            <el-input v-model="materialForm.name" placeholder="请输入原料名称" />
          </el-form-item>
        </el-col>
        <el-col :span="12">
          <el-form-item label="原料类型" prop="type">
            <el-select v-model="materialForm.type" placeholder="请选择原料类型"
style="width: 100%;">
              <el-option label="茶叶" value="tea" />
              <el-option label="糖类" value="sugar" />
              <el-option label="菌种 SCOBY" value="scooby" />
              <el-option label="调味原料" value="flavor" />
              <el-option label="包装材料" value="packaging" />
            </el-select>
          </el-form-item>
        </el-col>
      </el-row>
      <el-row :gutter="20">
        <el-col :span="12">
          <el-form-item label="产地" prop="origin">
```

```
      <el-input v-model="materialForm.origin" placeholder="请输入产地" />
    </el-form-item>
  </el-col>
  <el-col :span="12">
    <el-form-item label="品种" prop="variety">
      <el-input v-model="materialForm.variety" placeholder="请输入品种" />
    </el-form-item>
  </el-col>
</el-row>
<el-row :gutter="20">
  <el-col :span="12">
    <el-form-item label="等级" prop="grade">
      <el-input v-model="materialForm.grade" placeholder="请输入等级" />
    </el-form-item>
  </el-col>
  <el-col :span="12">
    <el-form-item label="供应商" prop="supplierId">
      <el-select
        v-model="materialForm.supplierId"
        placeholder="请选择供应商"
        filterable
        style="width: 100%;"
      >
        <el-option
          v-for="supplier in supplierList"
          :key="supplier.id"
          :label="supplier.name"
          :value="supplier.id"
        />
      </el-select>
    </el-form-item>
  </el-col>
</el-row>
<el-row :gutter="20">
  <el-col :span="12">
    <el-form-item label="采购日期" prop="purchaseDate">
      <el-date-picker
        v-model="materialForm.purchaseDate"
        type="date"
        placeholder="选择日期"
        style="width: 100%;"
        value-format="YYYY-MM-DD"
      />
    </el-form-item>
  </el-col>
  <el-col :span="12">
    <el-form-item label="保质期" prop="shelfLife">
      <el-input-number
        v-model="materialForm.shelfLife"
        :min="1"
```

```
        :max="3650"
        placeholder="请输入保质期天数"
        style="width: 100%;"
      />
      <span style="margin-left: 10px;">天</span>
    </el-form-item>
  </el-col>
</el-row>
<el-row :gutter="20">
  <el-col :span="12">
    <el-form-item label="单价" prop="unitPrice">
      <el-input-number
        v-model="materialForm.unitPrice"
        :min="0"
        :precision="2"
        placeholder="请输入单价"
        style="width: 100%;"
      />
      <span style="margin-left: 10px;">元</span>
    </el-form-item>
  </el-col>
  <el-col :span="12">
    <el-form-item label="单位" prop="unit">
      <el-input v-model="materialForm.unit" placeholder="如：kg、L、个" />
    </el-form-item>
  </el-col>
</el-row>
<el-row :gutter="20">
  <el-col :span="12">
    <el-form-item label="初始库存" prop="stock">
      <el-input-number
        v-model="materialForm.stock"
        :min="0"
        :precision="2"
        placeholder="请输入初始库存"
        style="width: 100%;"
      />
    </el-form-item>
  </el-col>
  <el-col :span="12">
    <el-form-item label="安全库存" prop="safetyStock">
      <el-input-number
        v-model="materialForm.safetyStock"
        :min="0"
        :precision="2"
        placeholder="请输入安全库存"
        style="width: 100%;"
      />
    </el-form-item>
  </el-col>
```

```
      </el-row>
      <el-form-item label="储存条件" prop="storageCondition">
        <el-input v-model="materialForm.storageCondition" placeholder="如：常温干燥、
冷藏" />
      </el-form-item>
      <el-form-item label="备注" prop="remark">
        <el-input
          v-model="materialForm.remark"
          type="textarea"
          :rows="3"
          placeholder="请输入备注信息"
        />
      </el-form-item>
    </el-form>
    <template #footer>
      <el-button @click="dialogVisible = false">取消</el-button>
      <el-button type="primary" @click="handleSubmit" :loading="submitting">
        确定
      </el-button>
    </template>
  </el-dialog>
 </div>
</template>
<script setup>
import { ref, reactive, computed, onMounted } from 'vue'
import { getMaterials, getMaterialCategories, createMaterial, updateMaterial, deleteMaterial }
from '@/api/materials'
import { ElMessage, ElMessageBox } from 'element-plus'
import { Box, Sugar, Cherry, Orange, Paper, Refresh, Search, RefreshLeft } from '@element-
plus/icons-vue'
// 响应式数据
const loading = ref(false)
const submitting = ref(false)
const dialogVisible = ref(false)
const dialogTitle = computed(() => isEdit.value ? '编辑原料' : '新增原料')
const isEdit = ref(false)
const currentMaterialId = ref('')
// 搜索表单
const searchForm = reactive({
  keyword: '',
  type: '',
  categoryId: ''
})
// 分页信息
const pagination = reactive({
  page: 1,
  size: 20,
  total: 0
})
// 原料列表
```

```
const materialList = ref([])
// 分类树数据
const categories = ref([])
const categoryTreeRef = ref(null)
const treeProps = {
  children: 'children',
  label: 'label'
}
// 原料表单
const materialFormRef = ref(null)
const materialForm = reactive({
  name: '',
  type: '',
  origin: '',
  variety: '',
  grade: '',
  supplierId: '',
  purchaseDate: '',
  shelfLife: 180,
  unitPrice: 0,
  unit: 'kg',
  stock: 0,
  safetyStock: 100,
  storageCondition: '常温干燥',
  remark: ''
})
// 表单验证规则
const formRules = {
  name: [
    { required: true, message: '请输入原料名称', trigger: 'blur' }
  ],
  type: [
    { required: true, message: '请选择原料类型', trigger: 'change' }
  ],
  supplierId: [
    { required: true, message: '请选择供应商', trigger: 'change' }
  ],
  purchaseDate: [
    { required: true, message: '请选择采购日期', trigger: 'change' }
  ],
  shelfLife: [
    { required: true, message: '请输入保质期', trigger: 'blur' }
  ],
  unitPrice: [
    { required: true, message: '请输入单价', trigger: 'blur' }
  ],
  unit: [
    { required: true, message: '请输入单位', trigger: 'blur' }
  ]
}
```

```
// 供应商列表（实际应从 API 获取）
const supplierList = ref([
  { id: 'SUP001', name: '某某茶业有限公司' },
  { id: 'SUP002', name: '某某糖业有限公司' },
  { id: 'SUP003', name: '某某农产品市场' }
])
// 方法：获取原料列表
const fetchMaterialList = async () => {
  loading.value = true
  try {
    const params = {
      page: pagination.page,
      size: pagination.size,
      keyword: searchForm.keyword,
      type: searchForm.type,
      categoryId: searchForm.categoryId
    }
    const response = await getMaterials(params)
    materialList.value = response.data.list
    pagination.total = response.data.total
  } catch (error) {
    ElMessage.error('获取原料列表失败')
  } finally {
    loading.value = false
  }
}
// 方法：获取分类树
const fetchCategories = async () => {
  try {
    const response = await getMaterialCategories()
    categories.value = response.data
  } catch (error) {
    ElMessage.error('获取分类失败')
  }
}
// 方法：分类节点点击事件
const handleCategoryClick = (data) => {
  searchForm.categoryId = data.id
  pagination.page = 1
  fetchMaterialList()
}
// 方法：刷新分类树
const handleRefreshCategories = () => {
  fetchCategories()
}
// 方法：搜索
const handleSearch = () => {
  pagination.page = 1
  fetchMaterialList()
}
```

```
// 方法：重置搜索
const handleReset = () => {
  searchForm.keyword = ''
  searchForm.type = ''
  searchForm.categoryId = ''
  categoryTreeRef.value?.setCurrentKey(null)
  pagination.page = 1
  fetchMaterialList()
}
// 方法：分页大小改变
const handleSizeChange = (size) => {
  pagination.size = size
  pagination.page = 1
  fetchMaterialList()
}
// 方法：当前页改变
const handleCurrentChange = (page) => {
  pagination.page = page
  fetchMaterialList()
}
// 方法：新增原料
const handleCreate = () => {
  isEdit.value = false
  resetForm()
  dialogVisible.value = true
}
// 方法：查看原料详情
const handleView = (row) => {
  ElMessageBox.alert(
    `
    <div style="line-height: 2;">
      <p><strong>原料编号：</strong>${row.materialId}</p>
      <p><strong>原料名称：</strong>${row.name}</p>
      <p><strong>类型：</strong>${getTypeLabel(row.type)}</p>
      <p><strong>产地：</strong>${row.origin}</p>
      <p><strong>供应商：</strong>${row.supplierName}</p>
      <p><strong>库存数量：</strong>${row.stock} ${row.unit}</p>
      <p><strong>单价：</strong>¥${row.unitPrice.toFixed(2)}</p>
      <p><strong>采购日期：</strong>${row.purchaseDate}</p>
      <p><strong>保质期：</strong>${row.shelfLife} 天</p>
      <p><strong>储存条件：</strong>${row.storageCondition}</p>
    </div>
    `,
    '原料详情',
    {
      dangerouslyUseHTMLString: true,
      confirmButtonText: '关闭'
    }
  )
}
```

```
// 方法：编辑原料
const handleEdit = (row) => {
  isEdit.value = true
  currentMaterialId.value = row.materialId
  Object.assign(materialForm, row)
  dialogVisible.value = true
}
// 方法：删除原料
const handleDelete = async (row) => {
  try {
    await ElMessageBox.confirm(
      `确定要删除原料"${row.name}"吗？此操作不可恢复。`,
      '警告',
      {
        confirmButtonText: '确定',
        cancelButtonText: '取消',
        type: 'warning'
      }
    )
    await deleteMaterial(row.materialId)
    ElMessage.success('删除成功')
    fetchMaterialList()
  } catch (error) {
    if (error !== 'cancel') {
      ElMessage.error('删除失败')
    }
  }
}
// 方法：提交表单
const handleSubmit = async () => {
  const valid = await materialFormRef.value?.validate()
  if (!valid) return
  submitting.value = true
  try {
    if (isEdit.value) {
      await updateMaterial(currentMaterialId.value, materialForm)
      ElMessage.success('更新成功')
    } else {
      await createMaterial(materialForm)
      ElMessage.success('创建成功')
    }
    dialogVisible.value = false
    fetchMaterialList()
  } catch (error) {
    ElMessage.error(isEdit.value ? '更新失败' : '创建失败')
  } finally {
    submitting.value = false
  }
}
// 方法：对话框关闭事件
```

```
const handleDialogClose = () => {
  materialFormRef.value?.resetFields()
  resetForm()
}
// 方法：重置表单
const resetForm = () => {
  Object.assign(materialForm, {
    name: '',
    type: '',
    origin: '',
    variety: '',
    grade: '',
    supplierId: '',
    purchaseDate: '',
    shelfLife: 180,
    unitPrice: 0,
    unit: 'kg',
    stock: 0,
    safetyStock: 100,
    storageCondition: '常温干燥',
    remark: ''
  })
}
// 方法：批量导入
const handleImport = () => {
  ElMessage.info('批量导入功能开发中')
}
// 方法：导出
const handleExport = async () => {
  ElMessage.info('导出功能开发中')
}
// 辅助方法：获取类型标签颜色
const getTypeTagColor = (type) => {
  const colorMap = {
    tea: 'success',
    sugar: 'warning',
    scooby: 'danger',
    flavor: 'info',
    packaging: ''
  }
  return colorMap[type] || ''
}
// 辅助方法：获取类型标签文本
const getTypeLabel = (type) => {
  const labelMap = {
    tea: '茶叶',
    sugar: '糖类',
    scooby: '菌种 SCOBY',
    flavor: '调味原料',
    packaging: '包装材料'
```

```
  }
  return labelMap[type] || type
}
// 辅助方法：获取状态标签颜色
const getStatusTagColor = (status) => {
  const colorMap = {
    normal: 'success',
    expired: 'danger',
    recalled: 'warning'
  }
  return colorMap[status] || ''
}
// 辅助方法：获取状态标签文本
const getStatusLabel = (status) => {
  const labelMap = {
    normal: '正常',
    expired: '已过期',
    recalled: '已召回'
  }
  return labelMap[status] || status
}
// 辅助方法：获取库存样式类
const getStockClass = (stock, safetyStock) => {
  if (stock <= 0) return 'stock-zero'
  if (stock < safetyStock) return 'stock-low'
  return ''
}
// 辅助方法：获取分类图标
const getCategoryIcon = (type) => {
  const iconMap = {
    tea: Box,
    sugar: Sugar,
    scooby: Cherry,
    flavor: Orange,
    packaging: Paper
  }
  return iconMap[type] || Box
}
// 生命周期：组件挂载
onMounted(() => {
  fetchMaterialList()
  fetchCategories()
})
</script>
<style scoped>
.materials-page {
  height: 100%;
}
.page-header {
  display: flex;
```

```
  justify-content: space-between;
  align-items: center;
  margin-bottom: 20px;
}
.page-title {
  font-size: 20px;
  font-weight: 500;
  color: #333;
}
.header-actions {
  display: flex;
  gap: 10px;
}
.category-card {
  height: calc(100vh - 180px);
}
.card-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.list-card {
  min-height: calc(100vh - 180px);
}
.tree-node {
  display: flex;
  align-items: center;
  gap: 5px;
  flex: 1;
  font-size: 14px;
}
.tree-node .count {
  margin-left: auto;
  color: #909399;
  font-size: 12px;
}
.search-bar {
  display: flex;
  align-items: center;
  flex-wrap: wrap;
  gap: 10px;
}
.stock-zero {
  color: #F56C6C;
  font-weight: bold;
}
.stock-low {
  color: #E6A23C;
  font-weight: bold;
}
```

```
</style>
<template>
  <div class="fermentation-page">
    <!-- 页面头部 -->
    <div class="page-header">
      <h2 class="page-title">发酵流程监控</h2>
      <el-button type="primary" @click="handleCreate">
        <el-icon><Plus /></el-icon> 创建发酵批次
      </el-button>
    </div>
    <!-- 批次卡片列表 -->
    <el-row :gutter="20" class="batch-cards">
      <el-col
        v-for="batch in batchList"
        :key="batch.batchId"
        :span="8"
        style="margin-bottom: 20px;"
      >
        <el-card
          class="batch-card"
          :class="getBatchCardClass(batch.status)"
          @click="handleViewBatch(batch)"
        >
          <template #header>
            <div class="card-header">
              <span class="batch-id">{{ batch.batchId }}</span>
              <el-tag :type="getStatusTagType(batch.status)">
                {{ getStatusLabel(batch.status) }}
              </el-tag>
            </div>
          </template>
          <div class="batch-info">
            <div class="info-row">
              <span class="label">发酵天数：</span>
              <span class="value">第 {{ batch.currentDay }} 天 / {{ batch.targetDays }} 天
</span>
            </div>
            <div class="info-row">
              <span class="label">当前温度：</span>
              <span class="value" :class="getTempClass(batch.currentTemp,
batch.targetTempMin, batch.targetTempMax)">
                {{ batch.currentTemp }}°C
              </span>
            </div>
            <div class="info-row">
              <span class="label">当前 pH 值：</span>
              <span class="value" :class="getPhClass(batch.currentPh, batch.targetPhMin,
batch.targetPhMax)">
                {{ batch.currentPh }}
              </span>
```

```
          </div>
          <div class="info-row">
            <span class="label">成熟度评分：</span>
            <span class="value" :class="getMaturityClass(batch.maturityScore)">
              {{ batch.maturityScore }}分
            </span>
          </div>
        </div>
        <div class="batch-progress">
          <el-progress
            :percentage="getProgressPercentage(batch.currentDay, batch.targetDays)"
            :color="getProgressColor(batch.maturityScore)"
            :stroke-width="8"
          />
        </div>
        <div class="batch-actions">
          <el-button link type="primary" size="small"
@click.stop="handleRecordData(batch)">
            录入数据
          </el-button>
          <el-button link type="primary" size="small"
@click.stop="handleViewChart(batch)">
            查看曲线
          </el-button>
        </div>
      </el-card>
    </el-col>
  </el-row>
  <!-- 数据录入对话框 -->
  <el-dialog
    v-model="recordDialogVisible"
    title="每日监控数据录入"
    width="600px"
  >
    <el-form
      ref="recordFormRef"
      :model="recordForm"
      :rules="recordFormRules"
      label-width="120px"
    >
      <div class="batch-summary">
        <span>批次编号：{{ currentBatch.batchId }}</span>
        <span style="margin-left: 20px;">发酵天数：第 {{ currentBatch.currentDay }} 天
</span>
        <span style="margin-left: 20px;">成熟度：{{ currentBatch.maturityScore }}分</span>
      </div>
      <el-divider />
      <el-tabs v-model="activeTab">
        <el-tab-pane label="手动录入" name="manual">
          <el-form-item label="录入日期" prop="recordDate">
```

```
  <el-date-picker
    v-model="recordForm.recordDate"
    type="date"
    placeholder="选择日期"
    style="width: 100%;"
    value-format="YYYY-MM-DD"
  />
</el-form-item>
<el-form-item label="录入时间" prop="recordTime">
  <el-time-picker
    v-model="recordForm.recordTime"
    placeholder="选择时间"
    style="width: 100%;"
    value-format="HH:mm"
  />
</el-form-item>
<el-form-item label="温度 (°C)" prop="temperature">
  <el-input-number
    v-model="recordForm.temperature"
    :min="0"
    :max="50"
    :precision="1"
    :step="0.1"
    style="width: 100%;"
  />
</el-form-item>
<el-form-item label="湿度 (%)" prop="humidity">
  <el-input-number
    v-model="recordForm.humidity"
    :min="0"
    :max="100"
    :precision="0"
    style="width: 100%;"
  />
</el-form-item>
<el-form-item label="pH 值" prop="ph">
  <el-input-number
    v-model="recordForm.ph"
    :min="0"
    :max="14"
    :precision="1"
    :step="0.1"
    style="width: 100%;"
  />
</el-form-item>
<el-form-item label="气味" prop="odor">
  <el-radio-group v-model="recordForm.odor">
    <el-radio label="normal">正常</el-radio>
    <el-radio label="too_sour">酸味过重</el-radio>
    <el-radio label="abnormal">异常</el-radio>
```

```
    </el-radio-group>
  </el-form-item>
  <el-form-item label="颜色" prop="color">
    <el-radio-group v-model="recordForm.color">
      <el-radio label="golden">金黄</el-radio>
      <el-radio label="dark_brown">深褐</el-radio>
      <el-radio label="cloudy">浑浊</el-radio>
    </el-radio-group>
  </el-form-item>
  <el-form-item label="菌膜厚度" prop="filmThickness">
    <el-radio-group v-model="recordForm.filmThickness">
      <el-radio label="thin">薄</el-radio>
      <el-radio label="medium">适中</el-radio>
      <el-radio label="thick">厚</el-radio>
    </el-radio-group>
  </el-form-item>
  <el-form-item label="菌膜覆盖率" prop="filmCoverage">
    <el-radio-group v-model="recordForm.filmCoverage">
      <el-radio label="low">&lt;50%</el-radio>
      <el-radio label="medium">50%-80%</el-radio>
      <el-radio label="high">&gt;80%</el-radio>
    </el-radio-group>
  </el-form-item>
</el-tab-pane>
<el-tab-pane label="自动采集" name="auto">
  <div class="auto-data-panel">
    <el-descriptions :column="2" border>
      <el-descriptions-item label="设备编号">
        {{ deviceData.deviceId || '-' }}
      </el-descriptions-item>
      <el-descriptions-item label="设备状态">
        <el-tag :type="deviceData.online ? 'success' : 'danger'">
          {{ deviceData.online ? '在线' : '离线' }}
        </el-tag>
      </el-descriptions-item>
      <el-descriptions-item label="当前温度">
        {{ deviceData.temperature }}°C
      </el-descriptions-item>
      <el-descriptions-item label="更新时间">
        {{ deviceData.updateTime || '-' }}
      </el-descriptions-item>
      <el-descriptions-item label="当前湿度">
        {{ deviceData.humidity }}%
      </el-descriptions-item>
      <el-descriptions-item label="数据来源">
        <el-tag type="info">自动采集</el-tag>
      </el-descriptions-item>
    </el-descriptions>
    <div style="margin-top: 20px; text-align: center;">
      <el-button type="primary" @click="handleFetchDeviceData">
```

```
            <el-icon><Refresh /></el-icon> 刷新数据
          </el-button>
        </div>
      </div>
    </el-tab-pane>
  </el-tabs>
</el-form>
<template #footer>
  <el-button @click="recordDialogVisible = false">取消</el-button>
  <el-button type="primary" @click="handleSubmitRecord" :loading="submitting">
    保存记录
  </el-button>
</template>
</el-dialog>
</div>
</template>
<script setup>
import { ref, reactive, onMounted, onUnmounted } from 'vue'
import { useRouter } from 'vue-router'
import { getFermentationBatches, recordFermentationData, getRealtimeData } from
'@/api/fermentation'
import { ElMessage } from 'element-plus'
import { Plus, Refresh } from '@element-plus/icons-vue'
const router = useRouter()
// 响应式数据
const batchList = ref([])
const loading = ref(false)
const recordDialogVisible = ref(false)
const submitting = ref(false)
const activeTab = ref('manual')
const currentBatch = ref({})
const deviceData = ref({})
// 录入表单
const recordFormRef = ref(null)
const recordForm = reactive({
  recordDate: new Date().toISOString().split('T')[0],
  recordTime: new Date().toTimeString().slice(0, 5),
  temperature: 25.0,
  humidity: 65,
  ph: 4.2,
  odor: 'normal',
  color: 'golden',
  filmThickness: 'medium',
  filmCoverage: 'high'
})
// 表单验证规则
const recordFormRules = {
  recordDate: [
    { required: true, message: '请选择录入日期', trigger: 'change' }
  ],
```

```
  recordTime: [
    { required: true, message: '请选择录入时间', trigger: 'change' }
  ],
  temperature: [
    { required: true, message: '请输入温度', trigger: 'blur' }
  ],
  ph: [
    { required: true, message: '请输入 pH 值', trigger: 'blur' }
  ]
}
// 方法：获取批次列表
const fetchBatchList = async () => {
  loading.value = true
  try {
    const response = await getFermentationBatches({
      page: 1,
      size: 50,
      status: 'fermenting'
    })
    batchList.value = response.data.list
  } catch (error) {
    ElMessage.error('获取批次列表失败')
  } finally {
    loading.value = false
  }
}
// 方法：创建新批次
const handleCreate = () => {
  router.push('/fermentation/create')
}
// 方法：查看批次详情
const handleViewBatch = (batch) => {
  router.push(`/fermentation/${batch.batchId}`)
}
// 方法：录入数据
const handleRecordData = (batch) => {
  currentBatch.value = batch
  recordDialogVisible.value = true
  // 如果批次绑定了设备，获取设备数据
  if (batch.deviceId) {
    fetchDeviceData(batch.deviceId)
  }
}
// 方法：查看曲线
const handleViewChart = (batch) => {
  router.push(`/fermentation/${batch.batchId}?tab=chart`)
}
// 方法：获取设备数据
const fetchDeviceData = async (deviceId) => {
  try {
```

```
    const batchId = currentBatch.value.batchId
    const response = await getRealtimeData(batchId)
    deviceData.value = response.data
  } catch (error) {
    console.error('获取设备数据失败:', error)
  }
}
// 方法：手动刷新设备数据
const handleFetchDeviceData = () => {
  if (currentBatch.value.deviceId) {
    fetchDeviceData(currentBatch.value.deviceId)
    ElMessage.success('数据已刷新')
  }
}
// 方法：提交记录
const handleSubmitRecord = async () => {
  const valid = await recordFormRef.value?.validate()
  if (!valid) return
  submitting.value = true
  try {
    const data = {
      batchId: currentBatch.value.batchId,
      ...recordForm,
      dataSource: activeTab.value === 'auto' ? 'auto' : 'manual'
    }
    await recordFermentationData(currentBatch.value.batchId, data)
    ElMessage.success('保存成功')
    recordDialogVisible.value = false
    fetchBatchList()
  } catch (error) {
    ElMessage.error('保存失败')
  } finally {
    submitting.value = false
  }
}
// 辅助方法：获取批次卡片样式类
const getBatchCardClass = (status) => {
  const classMap = {
    fermenting: 'status-fermenting',
    completed: 'status-completed',
    failed: 'status-failed'
  }
  return classMap[status] || ''
}
// 辅助方法：获取状态标签类型
const getStatusTagType = (status) => {
  const typeMap = {
    fermenting: 'warning',
    completed: 'success',
    failed: 'danger'
```

```
  }
  return typeMap[status] || ''
}
// 辅助方法：获取状态标签文本
const getStatusLabel = (status) => {
  const labelMap = {
    fermenting: '发酵中',
    completed: '已完成',
    failed: '已失败'
  }
  return labelMap[status] || status
}
// 辅助方法：获取进度百分比
const getProgressPercentage = (current, target) => {
  return Math.min(Math.round((current / target) * 100), 100)
}
// 辅助方法：获取进度条颜色
const getProgressColor = (score) => {
  if (score >= 80) return '#67C23A'
  if (score >= 60) return '#E6A23C'
  return '#F56C6C'
}
// 辅助方法：获取温度样式类
const getTempClass = (current, min, max) => {
  if (current < min || current > max) return 'value-abnormal'
  return 'value-normal'
}
// 辅助方法：获取 pH 值样式类
const getPhClass = (current, min, max) => {
  if (current < min || current > max) return 'value-abnormal'
  return 'value-normal'
}
// 辅助方法：获取成熟度样式类
const getMaturityClass = (score) => {
  if (score >= 80) return 'value-excellent'
  if (score >= 60) return 'value-good'
  return 'value-poor'
}
// 定时刷新数据
let refreshTimer = null
// 生命周期：组件挂载
onMounted(() => {
  fetchBatchList()
  // 每 30 秒自动刷新一次数据
  refreshTimer = setInterval(() => {
    fetchBatchList()
  }, 30000)
})
// 生命周期：组件卸载
onUnmounted(() => {
```

```
  if (refreshTimer) {
    clearInterval(refreshTimer)
  }
})
</script>
<style scoped>
.fermentation-page {
  height: 100%;
}
.page-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 20px;
}
.page-title {
  font-size: 20px;
  font-weight: 500;
  color: #333;
}
.batch-cards {
  margin-top: 20px;
}
.batch-card {
  cursor: pointer;
  transition: all 0.3s;
}
.batch-card:hover {
  box-shadow: 0 4px 16px rgba(0, 0, 0, 0.12);
  transform: translateY(-2px);
}
.batch-card.status-fermenting {
  border-left: 4px solid #409EFF;
}
.batch-card.status-completed {
  border-left: 4px solid #67C23A;
}
.batch-card.status-failed {
  border-left: 4px solid #F56C6C;
}
.card-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.batch-id {
  font-size: 16px;
  font-weight: 500;
  color: #333;
}
```

```
.batch-info {
  margin: 15px 0;
}
.info-row {
  display: flex;
  justify-content: space-between;
  margin-bottom: 10px;
  font-size: 14px;
}
.info-row .label {
  color: #909399;
}
.info-row .value {
  font-weight: 500;
  color: #333;
}
.value-normal {
  color: #67C23A;
}
.value-abnormal {
  color: #F56C6C;
}
.value-excellent {
  color: #67C23A;
  font-weight: bold;
}
.value-good {
  color: #E6A23C;
}
.value-poor {
  color: #F56C6C;
}
.batch-progress {
  margin: 15px 0;
}
.batch-actions {
  display: flex;
  justify-content: space-around;
  border-top: 1px solid #ebeef5;
  padding-top: 10px;
}
.batch-summary {
  padding: 10px;
  background-color: #f5f7fa;
  border-radius: 4px;
  margin-bottom: 15px;
  font-size: 14px;
  color: #606266;
}
.auto-data-panel {
```

```
  padding: 20px;
}
</style>
<template>
  <div class="monitoring-dashboard">
    <!-- 顶部信息栏 -->
    <div class="dashboard-header">
      <div class="header-title">实时监控大屏</div>
      <div class="header-time">{{ currentTime }}</div>
    </div>
    <!-- 关键指标卡片 -->
    <el-row :gutter="20" class="stats-row">
      <el-col :span="6">
        <div class="stat-card">
          <div class="stat-icon" style="background: linear-gradient(135deg, #667eea 0%,
#764ba2 100%);">
            <el-icon :size="32"><Monitor /></el-icon>
          </div>
          <div class="stat-content">
            <div class="stat-label">发酵中批次</div>
            <div class="stat-value">{{ stats.fermenting }}</div>
          </div>
        </div>
      </el-col>
      <el-col :span="6">
        <div class="stat-card">
          <div class="stat-icon" style="background: linear-gradient(135deg, #f093fb 0%,
#f5576c 100%);">
            <el-icon :size="32"><Warning /></el-icon>
          </div>
          <div class="stat-content">
            <div class="stat-label">今日报警</div>
            <div class="stat-value danger">{{ stats.alerts }}</div>
          </div>
        </div>
      </el-col>
      <el-col :span="6">
        <div class="stat-card">
          <div class="stat-icon" style="background: linear-gradient(135deg, #4facfe 0%,
#00f2fe 100%);">
            <el-icon :size="32"><Connection /></el-icon>
          </div>
          <div class="stat-content">
            <div class="stat-label">设备在线率</div>
            <div class="stat-value success">{{ stats.onlineRate }}%</div>
          </div>
        </div>
      </el-col>
      <el-col :span="6">
        <div class="stat-card">
```

```
        <div class="stat-icon" style="background: linear-gradient(135deg, #43e97b 0%,
#38f9d7 100%);">
          <el-icon :size="32"><TrendCharts /></el-icon>
        </div>
        <div class="stat-content">
          <div class="stat-label">平均成熟度</div>
          <div class="stat-value">{{ stats.avgMaturity }}</div>
        </div>
      </div>
    </el-col>
  </el-row>
  <!-- 图表区域 -->
  <el-row :gutter="20" class="charts-row">
    <el-col :span="12">
      <div class="chart-card">
        <div class="chart-header">
          <span>温度监控 (最近 24 小时)</span>
          <el-tag type="info">实时</el-tag>
        </div>
        <div ref="tempChartRef" class="chart-container"></div>
      </div>
    </el-col>
    <el-col :span="12">
      <div class="chart-card">
        <div class="chart-header">
          <span>pH 值监控 (最近 24 小时)</span>
          <el-tag type="info">实时</el-tag>
        </div>
        <div ref="phChartRef" class="chart-container"></div>
      </div>
    </el-col>
  </el-row>
  <!-- 批次列表和报警信息 -->
  <el-row :gutter="20" class="bottom-row">
    <el-col :span="16">
      <div class="table-card">
        <div class="card-header">
          <span>发酵批次列表</span>
          <el-button link type="primary" @click="handleRefresh">
            <el-icon><Refresh /></el-icon> 刷新
          </el-button>
        </div>
        <el-table :data="batchList" stripe max-height="400">
          <el-table-column prop="batchId" label="批次编号" width="150" />
          <el-table-column label="天数" width="100" align="center">
            <template #default="{ row }">
              第{{ row.currentDay }}天
            </template>
          </el-table-column>
          <el-table-column label="温度" width="100" align="center">
```

```
          <template #default="{ row }">
            <span :class="getTempClass(row.currentTemp, row.targetTempMin,
row.targetTempMax)">
              {{ row.currentTemp }}°C
            </span>
          </template>
        </el-table-column>
        <el-table-column label="pH" width="80" align="center">
          <template #default="{ row }">
            {{ row.currentPh }}
          </template>
        </el-table-column>
        <el-table-column prop="maturityScore" label="成熟度" width="100"
align="center">
          <template #default="{ row }">
            {{ row.maturityScore }}分
          </template>
        </el-table-column>
        <el-table-column prop="status" label="状态" width="100">
          <template #default="{ row }">
            <el-tag :type="getStatusTagType(row.status)" size="small">
              {{ getStatusLabel(row.status) }}
            </el-tag>
          </template>
        </el-table-column>
        <el-table-column label="操作" width="100">
          <template #default="{ row }">
            <el-button link type="primary" size="small" @click="handleViewDetail(row)">
              详情
            </el-button>
          </template>
        </el-table-column>
      </el-table>
    </div>
  </el-col>
  <el-col :span="8">
    <div class="alert-card">
      <div class="card-header">
        <span>最新报警</span>
        <el-badge :value="alertCount" class="item">
          <el-icon :size="20" color="#F56C6C"><Bell /></el-icon>
        </el-badge>
      </div>
      <div class="alert-list">
        <div
          v-for="alert in alertList"
          :key="alert.id"
          class="alert-item"
          :class="`alert-${alert.level}`"
        >
```

```
            <div class="alert-time">{{ alert.time }}</div>
            <div class="alert-content">{{ alert.message }}</div>
          </div>
          <div v-if="alertList.length === 0" class="alert-empty">
            <el-icon :size="40" color="#909399"><SuccessFilled /></el-icon>
            <p>暂无报警信息</p>
          </div>
        </div>
      </div>
    </el-col>
  </el-row>
 </div>
</template>
<script setup>
import { ref, reactive, onMounted, onUnmounted } from 'vue'
import { useRouter } from 'vue-router'
import { getFermentationBatches, getAlerts } from '@/api/fermentation'
import * as echarts from 'echarts'
import { ElMessage } from 'element-plus'
import { Monitor, Warning, Connection, TrendCharts, Refresh, Bell, SuccessFilled } from
'@element-plus/icons-vue'
const router = useRouter()
// 响应式数据
const currentTime = ref('')
const stats = reactive({
  fermenting: 0,
  alerts: 0,
  onlineRate: 95,
  avgMaturity: 76
})
const batchList = ref([])
const alertList = ref([])
const alertCount = ref(0)
const tempChartRef = ref(null)
const phChartRef = ref(null)
let tempChart = null
let phChart = null
let refreshTimer = null
let clockTimer = null
// 方法：更新当前时间
const updateCurrentTime = () => {
  const now = new Date()
  currentTime.value = now.toLocaleString('zh-CN', {
    year: 'numeric',
    month: '2-digit',
    day: '2-digit',
    hour: '2-digit',
    minute: '2-digit',
    second: '2-digit'
  })
```

```
}
// 方法：获取统计数据
const fetchStats = async () => {
  try {
    const response = await getFermentationBatches({
      page: 1,
      size: 100,
      status: 'fermenting'
    })
    const batches = response.data.list
    batchList.value = batches
    // 计算统计数据
    stats.fermenting = batches.length
    stats.avgMaturity = batches.length > 0
      ? Math.round(batches.reduce((sum, b) => sum + b.maturityScore, 0) / batches.length)
      : 0
    // 更新图表数据
    updateCharts(batches)
  } catch (error) {
    console.error('获取统计数据失败:', error)
  }
}
// 方法：获取报警列表
const fetchAlerts = async () => {
  try {
    const response = await getAlerts({
      page: 1,
      size: 10,
      status: 'pending'
    })
    alertList.value = response.data.list
    alertCount.value = response.data.total
    stats.alerts = response.data.total
  } catch (error) {
    console.error('获取报警列表失败:', error)
  }
}
// 方法：刷新数据
const handleRefresh = () => {
  fetchStats()
  fetchAlerts()
  ElMessage.success('数据已刷新')
}
// 方法：查看批次详情
const handleViewDetail = (batch) => {
  router.push(`/fermentation/${batch.batchId}`)
}
// 方法：初始化温度图表
const initTempChart = () => {
  if (!tempChartRef.value) return
```

```
tempChart = echarts.init(tempChartRef.value)
const option = {
  backgroundColor: 'transparent',
  tooltip: {
    trigger: 'axis',
    axisPointer: {
      type: 'cross'
    }
  },
  grid: {
    left: '3%',
    right: '4%',
    bottom: '3%',
    containLabel: true
  },
  xAxis: {
    type: 'category',
    boundaryGap: false,
    data: [],
    axisLine: {
      lineStyle: {
        color: '#909399'
      }
    }
  },
  yAxis: {
    type: 'value',
    axisLine: {
      lineStyle: {
        color: '#909399'
      }
    },
    splitLine: {
      lineStyle: {
        color: '#ebeef5'
      }
    }
  },
  series: [
    {
      name: '温度',
      type: 'line',
      smooth: true,
      symbol: 'circle',
      symbolSize: 6,
      data: [],
      itemStyle: {
        color: '#409EFF'
      },
      areaStyle: {
```

```
          color: new echarts.graphic.LinearGradient(0, 0, 0, 1, [
            { offset: 0, color: 'rgba(64, 158, 255, 0.3)' },
            { offset: 1, color: 'rgba(64, 158, 255, 0.05)' }
          ])
        }
      }
    ]
  }
  tempChart.setOption(option)
}
// 方法：初始化 pH 图表
const initPhChart = () => {
  if (!phChartRef.value) return
  phChart = echarts.init(phChartRef.value)
  const option = {
    backgroundColor: 'transparent',
    tooltip: {
      trigger: 'axis',
      axisPointer: {
        type: 'cross'
      }
    },
    grid: {
      left: '3%',
      right: '4%',
      bottom: '3%',
      containLabel: true
    },
    xAxis: {
      type: 'category',
      boundaryGap: false,
      data: [],
      axisLine: {
        lineStyle: {
          color: '#909399'
        }
      }
    },
    yAxis: {
      type: 'value',
      axisLine: {
        lineStyle: {
          color: '#909399'
        }
      },
      splitLine: {
        lineStyle: {
          color: '#ebeef5'
        }
      }
```

```
    },
    series: [
      {
        name: 'pH 值',
        type: 'line',
        smooth: true,
        symbol: 'circle',
        symbolSize: 6,
        data: [],
        itemStyle: {
          color: '#E6A23C'
        },
        areaStyle: {
          color: new echarts.graphic.LinearGradient(0, 0, 0, 1, [
            { offset: 0, color: 'rgba(230, 162, 60, 0.3)' },
            { offset: 1, color: 'rgba(230, 162, 60, 0.05)' }
          ])
        }
      }
    ]
  }
  phChart.setOption(option)
}
// 方法：更新图表数据
const updateCharts = (batches) => {
  if (batches.length === 0) return
  // 生成时间轴（最近 24 小时）
  const now = new Date()
  const timeLabels = []
  for (let i = 23; i >= 0; i--) {
    const time = new Date(now - i * 3600000)
    timeLabels.push(time.getHours().toString().padStart(2, '0') + ':00')
  }
  // 模拟温度数据（取第一个批次的温度）
  const tempData = []
  const phData = []
  let baseTemp = 25
  let basePh = 4.0
  for (let i = 0; i < 24; i++) {
    baseTemp += (Math.random() - 0.5) * 0.5
    basePh -= 0.02
    tempData.push(parseFloat(baseTemp.toFixed(1)))
    phData.push(parseFloat(basePh.toFixed(1)))
  }
  // 更新温度图表
  if (tempChart) {
    tempChart.setOption({
      xAxis: {
        data: timeLabels
      },
```

```
      series: [{
        data: tempData
      }]
    })
  }
  // 更新 pH 图表
  if (phChart) {
    phChart.setOption({
      xAxis: {
        data: timeLabels
      },
      series: [{
        data: phData
      }]
    })
  }
}
// 辅助方法：获取温度样式类
const getTempClass = (current, min, max) => {
  if (current < min || current > max) return 'temp-abnormal'
  return 'temp-normal'
}
// 辅助方法：获取状态标签类型
const getStatusTagType = (status) => {
  const typeMap = {
    fermenting: 'warning',
    completed: 'success',
    failed: 'danger'
  }
  return typeMap[status] || ''
}
// 辅助方法：获取状态标签文本
const getStatusLabel = (status) => {
  const labelMap = {
    fermenting: '发酵中',
    completed: '已完成',
    failed: '已失败'
  }
  return labelMap[status] || status
}
// 生命周期：组件挂载
onMounted(() => {
  updateCurrentTime()
  fetchStats()
  fetchAlerts()
  // 初始化图表
  initTempChart()
  initPhChart()
  // 定时更新时间（每秒）
  clockTimer = setInterval(updateCurrentTime, 1000)
```

```
  // 定时刷新数据（每 30 秒）
  refreshTimer = setInterval(() => {
    fetchStats()
    fetchAlerts()
  }, 30000)
  // 监听窗口大小变化，重绘图表
  window.addEventListener('resize', () => {
    tempChart?.resize()
    phChart?.resize()
  })
})
// 生命周期：组件卸载
onUnmounted(() => {
  if (refreshTimer) {
    clearInterval(refreshTimer)
  }
  if (clockTimer) {
    clearInterval(clockTimer)
  }
  tempChart?.dispose()
  phChart?.dispose()
  window.removeEventListener('resize', () => {})
})
</script>
<style scoped>
.monitoring-dashboard {
  background: linear-gradient(135deg, #0B1120 0%, #1a2332 100%);
  min-height: calc(100vh - 80px);
  padding: 20px;
  color: white;
}
.dashboard-header {
  text-align: center;
  margin-bottom: 30px;
}
.header-title {
  font-size: 28px;
  font-weight: bold;
  margin-bottom: 10px;
}
.header-time {
  font-size: 16px;
  color: #a0aec0;
}
.stats-row {
  margin-bottom: 20px;
}
.stat-card {
  background: rgba(255, 255, 255, 0.1);
  border-radius: 8px;
```

```
  padding: 20px;
  display: flex;
  align-items: center;
  gap: 15px;
  backdrop-filter: blur(10px);
  border: 1px solid rgba(255, 255, 255, 0.1);
}
.stat-icon {
  width: 60px;
  height: 60px;
  border-radius: 12px;
  display: flex;
  align-items: center;
  justify-content: center;
  color: white;
}
.stat-content {
  flex: 1;
}
.stat-label {
  font-size: 14px;
  color: #a0aec0;
  margin-bottom: 5px;
}
.stat-value {
  font-size: 28px;
  font-weight: bold;
  color: white;
}
.stat-value.danger {
  color: #F56C6C;
}
.stat-value.success {
  color: #67C23A;
}
.charts-row {
  margin-bottom: 20px;
}
.chart-card {
  background: rgba(255, 255, 255, 0.05);
  border-radius: 8px;
  padding: 20px;
  border: 1px solid rgba(255, 255, 255, 0.1);
}
.chart-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 15px;
  font-size: 16px;
```

```
  font-weight: 500;
}
.chart-container {
  width: 100%;
  height: 300px;
}
.bottom-row {
  margin-bottom: 20px;
}
.table-card,
.alert-card {
  background: rgba(255, 255, 255, 0.05);
  border-radius: 8px;
  padding: 20px;
  border: 1px solid rgba(255, 255, 255, 0.1);
  max-height: 500px;
}
.card-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 15px;
  font-size: 16px;
  font-weight: 500;
}
.temp-normal {
  color: #67C23A;
}
.temp-abnormal {
  color: #F56C6C;
  font-weight: bold;
}
.alert-list {
  max-height: 400px;
  overflow-y: auto;
}
.alert-item {
  padding: 10px;
  margin-bottom: 8px;
  border-left: 3px solid #F56C6C;
  background: rgba(255, 255, 255, 0.05);
  border-radius: 4px;
  font-size: 14px;
}
.alert-item.alert-high {
  border-left-color: #F56C6C;
  background: rgba(245, 108, 108, 0.1);
}
.alert-item.alert-medium {
  border-left-color: #E6A23C;
```

```
  background: rgba(230, 162, 60, 0.1);
}
.alert-item.alert-low {
  border-left-color: #909399;
  background: rgba(144, 147, 153, 0.1);
}
.alert-time {
  font-size: 12px;
  color: #a0aec0;
  margin-bottom: 5px;
}
.alert-content {
  color: white;
}
.alert-empty {
  text-align: center;
  padding: 40px 0;
  color: #909399;
}
.alert-empty p {
  margin-top: 10px;
  font-size: 14px;
}
</style>
/**
 * 工具函数集合
 */
/**
 * 格式化日期时间
 * @param {Date|String} date - 日期对象或日期字符串
 * @param {String} format - 格式字符串，默认 'YYYY-MM-DD HH:mm:ss'
 * @returns {String} 格式化后的日期字符串
 */
export function formatDate(date, format = 'YYYY-MM-DD HH:mm:ss') {
  if (!date) return ''
  const d = new Date(date)
  if (isNaN(d.getTime())) return ''
  const year = d.getFullYear()
  const month = String(d.getMonth() + 1).padStart(2, '0')
  const day = String(d.getDate()).padStart(2, '0')
  const hours = String(d.getHours()).padStart(2, '0')
  const minutes = String(d.getMinutes()).padStart(2, '0')
  const seconds = String(d.getSeconds()).padStart(2, '0')
  return format
    .replace('YYYY', year)
    .replace('MM', month)
    .replace('DD', day)
    .replace('HH', hours)
    .replace('mm', minutes)
    .replace('ss', seconds)
```

```
}
/**
 * 格式化文件大小
 * @param {Number} bytes - 文件大小（字节）
 * @returns {String} 格式化后的文件大小
 */
export function formatFileSize(bytes) {
  if (bytes === 0) return '0 B'
  const k = 1024
  const sizes = ['B', 'KB', 'MB', 'GB', 'TB']
  const i = Math.floor(Math.log(bytes) / Math.log(k))
  return (bytes / Math.pow(k, i)).toFixed(2) + ' ' + sizes[i]
}
/**
 * 深拷贝对象
 * @param {Object} obj - 要拷贝的对象
 * @returns {Object} 拷贝后的新对象
 */
export function deepClone(obj) {
  if (obj === null || typeof obj !== 'object') return obj
  if (obj instanceof Date) return new Date(obj)
  if (obj instanceof Array) return obj.map(item => deepClone(item))
  const clonedObj = {}
  for (const key in obj) {
    if (obj.hasOwnProperty(key)) {
      clonedObj[key] = deepClone(obj[key])
    }
  }
  return clonedObj
}
/**
 * 防抖函数
 * @param {Function} func - 要防抖的函数
 * @param {Number} delay - 延迟时间（毫秒）
 * @returns {Function} 防抖后的函数
 */
export function debounce(func, delay = 300) {
  let timer = null
  return function (...args) {
    if (timer) clearTimeout(timer)
    timer = setTimeout(() => {
      func.apply(this, args)
    }, delay)
  }
}
/**
 * 节流函数
 * @param {Function} func - 要节流的函数
 * @param {Number} delay - 延迟时间（毫秒）
 * @returns {Function} 节流后的函数
```

```
 */
export function throttle(func, delay = 300) {
  let timer = null
  return function (...args) {
    if (!timer) {
      timer = setTimeout(() => {
        func.apply(this, args)
        timer = null
      }, delay)
    }
  }
}
/**
 * 生成唯一 ID
 * @param {String} prefix - ID 前缀
 * @returns {String} 唯一 ID
 */
export function generateId(prefix = '') {
  const timestamp = Date.now().toString(36)
  const random = Math.random().toString(36).substring(2, 9)
  return prefix ? `${prefix}_${timestamp}${random}` : `${timestamp}${random}`
}
/**
 * 下载文件
 * @param {Blob} blob - 文件 Blob 对象
 * @param {String} filename - 文件名
 */
export function downloadFile(blob, filename) {
  const url = window.URL.createObjectURL(blob)
  const link = document.createElement('a')
  link.href = url
  link.download = filename
  document.body.appendChild(link)
  link.click()
  document.body.removeChild(link)
  window.URL.revokeObjectURL(url)
}
/**
 * 数字格式化
 * @param {Number} num - 要格式化的数字
 * @param {Number} decimals - 小数位数
 * @returns {String} 格式化后的数字字符串
 */
export function formatNumber(num, decimals = 2) {
  if (isNaN(num)) return ''
  return Number(num).toFixed(decimals)
}
/**
 * 计算百分比
 * @param {Number} value - 当前值
```

```
 * @param {Number} total - 总值
 * @param {Number} decimals - 小数位数
 * @returns {String} 百分比字符串
 */
export function calculatePercentage(value, total, decimals = 2) {
  if (total === 0) return '0%'
  return ((value / total) * 100).toFixed(decimals) + '%'
}
/**
 * 验证邮箱格式
 * @param {String} email - 邮箱地址
 * @returns {Boolean} 是否有效
 */
export function isValidEmail(email) {
  const re = /^[^\s@]+@[^\s@]+\.[^\s@]+$/
  return re.test(email)
}
/**
 * 验证手机号格式
 * @param {String} phone - 手机号
 * @returns {Boolean} 是否有效
 */
export function isValidPhone(phone) {
  const re = /^1[3-9]\d{9}$/
  return re.test(phone)
}
/**
 * 获取随机颜色
 * @returns {String} 随机颜色值（十六进制）
 */
export function getRandomColor() {
  const letters = '0123456789ABCDEF'
  let color = '#'
  for (let i = 0; i < 6; i++) {
    color += letters[Math.floor(Math.random() * 16)]
  }
  return color
}
/**
 * 全局样式文件
 */
/* 重置默认样式 */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
html,
body {
  width: 100%;
```

```
  height: 100%;
  font-family: 'Microsoft YaHei', 'PingFang SC', 'Helvetica Neue', Helvetica, Arial, sans-serif;
  font-size: 14px;
  color: #333;
  background-color: #f0f2f5;
}
/* 滚动条样式 */
::-webkit-scrollbar {
  width: 8px;
  height: 8px;
}
::-webkit-scrollbar-track {
  background: #f1f1f1;
}
::-webkit-scrollbar-thumb {
  background: #888;
  border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
  background: #555;
}
/* 链接样式 */
a {
  color: #409EFF;
  text-decoration: none;
  transition: color 0.3s;
}
a:hover {
  color: #66b1ff;
}
/* 表格样式 */
.el-table {
  font-size: 14px;
}
.el-table th {
  background-color: #f5f7fa;
  font-weight: 500;
}
/* 表单样式 */
.el-form-item__label {
  font-weight: 500;
}
/* 按钮样式 */
.el-button {
  font-weight: 500;
}
/* 对话框样式 */
.el-dialog__header {
  padding: 20px 20px 10px;
  border-bottom: 1px solid #ebeef5;
```

```
}
.el-dialog__body {
  padding: 20px;
}
.el-dialog__footer {
  padding: 10px 20px 20px;
  border-top: 1px solid #ebeef5;
}
/* 消息提示样式 */
.el-message {
  min-width: 300px;
}
/* 标签样式 */
.el-tag {
  font-weight: 500;
}
/* 进度条样式 */
.el-progress__text {
  font-weight: 500;
}
```